

Посібник з налаштування та розгортання

Завантажте мережу без міток, підключіть телеметрію та зворотний зв'язок операторів і запустіть рекомендаційний скорер у продакшені

Виробничий регламент: як запускатч Entry завантажує згенеровану збірку, як налаштувати входи телеметрії та канал зворотного зв'язку, як рекомендації виходять із системи, як безперервне навчання працює без переустановлення та які шлюзи безпеки слід зберігати.

Документ	Посібник з налаштування та розгортання
Продукт	Jneopallium Self-Supervised Maintenance Guardian
Точка входу	com.rakovpublic.jneuropallium.worker.application.Entry
Збірка	model/self-supervised-maintenance
Режими	local · http · grpc
Безпека	ADVISORY / лише читання
Автор	Дмитро Раковський — Харків, Україна
Дата	2 липня 2026
Ліцензія	BSD 3-Clause

Зміст

1. Що ви розгортаєте
2. Передумови
3. Крок 1 — збірка та IContext
4. Крок 2 — запустити точку входу
5. Крок 3 — підключити входи телеметрії
6. Крок 4 — підключити канал зворотного зв'язку
7. Крок 5 — куди йдуть рекомендації
8. Крок 6 — режими розгортання
9. Крок 7 — безперервне навчання у продакшені
10. Розкатка: тінь → рекомендація
11. Експлуатація та моніторинг
12. Контрольний список безпеки
13. Усунення несправностей

1 Що ви розгортаєте

Робочий вузол Jneopallium, що приймає телеметрію активів, оцінює кожен кадр через мережу обслуговування без міток і видає рекомендації лише для читання із сімейством несправності та запасом часу. Він також споживає зворотний зв'язок операторів і адаптує власні пороги тривоги на місці — без перезапуску, без переустановлення. Він ніколи не керує пристроєм.

Єдине правило, що ніколи не змінюється

Ця модель рекомендаційна. Вона радить огляд; вона не планує роботи, не спрацьовує блокуванням і не рухає виконавчий механізм. Жорсткий шар безпеки лишається цілком поза навченим шляхом.

2 Передумови

- JDK 17 і зібраний артефакт worker (worker-1.0-SNAPSHOT.jar) або ваш упакований JAR моделі.
- Навчена збірка з посібника з навчання у model/self-supervised-maintenance/.
- Джерело телеметрії (OPC UA чи MQTT) та, для навчання, джерело зворотного зв'язку (подія UI/CMMS, яку позначає оператор).
- Записуване сховище для збереженого стану нейронів (щоб навчання переживало перезапуски).

3 Крок 1 — збірка та IContext

Розгортувана модель — це каталог збірки. production-context.json — це IContext, який десеріалізує запускар; його ключові записи:

Ключ IContext	Значення / зміст
configuration.input.layermeta	п'ять шарів (0–4)
neuronnet.classes	чотири класи нейронів ssmaint (перевіряються на старті)
input.inputs	джерело AssetTelemetrySignal (OPC UA / MQTT)
outputAggregator	куди записуються рекомендації (JSONL / Kafka / MQTT)
storage.json	model/self-supervised-maintenance (збережений стан)
processing.frequency.map / slowfast.ratio	каданс швидкої телеметрії проти повільного зв'язку
isteacherstudying=false, discriminatorsAmount=0, infiniteRun=true	цикл рекомендаційного висновку
safetyMode / advisoryOnly	ADVISORY / true — примусово

Запускар перевіряє neuronnet.classes на старті й швидко падає, якщо класу бракує — тож хибно налаштована збірка ніколи не стартує наполовину.

4 Крок 2 — запустити точку входу

```
java -cp worker-1.0-SNAPSHOT.jar \
  com.rakovpublic.jneuropallium.worker.application.Entry \
```

```
local <model-jar-url> <IContext-class> \
worker/src/main/resources/model/self-supervised-maintenance/production-context.json
```

Runner десеріалізує IContext, завантажує ваш JAR моделі через завантажувач класів JAR, обирає IApplication для вибраного режиму й починає цикл висновку. Далі — телеметрія на вході, рекомендації на виході.

5 Крок 3 — підключити входи телеметрії

Зіставте живі теги кожного активу з AssetTelemetrySignal на такт: вісім каналів сенсорів плюс цілочисельний режим роботи. Використовуйте стандартні промислові входи:

- **OPC UA** — OPCUaMeasurementInput для тегів процесу (тиск, витрата, температура, потужність, положення клапана).
- **MQTT / Sparkplug B** — MqttMetricInput для IIoT-шлюзів і потоків вібрації/акустики.
- Виведіть **режим** зі смуги навантаження чи потужності, якщо не публікуєте його напряму.

Правильно задайте назви сенсорів

Ваги відновлення прив'язані до назв сенсорів. Зіставлення тег→сенсор має точно збігатися з sensorOrder у шарі 1, інакше залишки безглузді. Перевірте зіставлення один раз при введенні в експлуатацію.

6 Крок 4 — підключити канал зворотного зв'язку

Саме це змушує модель вдосконалюватися. Коли оператор виносить вердикт щодо рекомендації у вашому UI, CMMS чи SIEM, опублікуйте OperatorFeedbackSignal, що несе: актив, сімейство несправності, підтверджено / хибне, особу оператора та значення зсуву домену, спостережене коли рекомендація спрацювала (щоб адаптер міг заморозитися під час новизни). Зворотний зв'язок працює на повільному циклі; він не мусить бути в реальному часі.

- **Підтверджені** рекомендації трохи послаблюють поріг цього сімейства.
- **Хибні** рекомендації підвищують його — обмежено й з обмеженням частоти.
- Слід приймати лише **автентифікований** зворотний зв'язок оператора; позначайте походження для аудиту.

7 Крок 5 — куди йдуть рекомендації

Рекомендації виходять через IOutputAggregator, який створює середовище виконання. Оберіть під майданчик:

Агрегатор	Використання
JsonlResultAggregator	прийом у файл/SIEM, найпростіше перше розгортання
KafkaAdvisoryOutputAggregator	потік у корпоративну шину / історіан
MqttAdvisoryOutputAggregator	публікація назад у IIoT-брокер / дашборд

Кожен MaintenanceAdvisorySignal несе актив, сімейство несправності, тяжкість, оцінений запас часу, невизначеність і зрозумілу людині рекомендацію. advisoryOnly завжди true на дроті.

8 Крок 6 — режими розгортання

Режим	Форма	Коли
local	один вузол в одному JVM	установка, комірка, крайовий вузол
http	майстер + HTTP-вузли	кілька ліній, централізована координація
grpc (client/master)	майстер + gRPC-вузли	більші парки, менша затримка

Модель однакова в усіх режимах; змінюються лише транспорт і топологія вузлів. Почніть з local, масштабуйтеся, коли кількість активів того вимагає.

9 Крок 7 — безперервне навчання у продакшені

Навчання під час роботи живе у стані нейронів і зберігається в storage.json, тож переживає перезапуски без перезбирання. Рекомендована дисципліна для розвитку чогось важчого за порogi — «чемпіон/претендент»:

- **чемпіон** (поточний, стабільний) видає рекомендації;
- **претендент** адаптується на зібраному зворотному зв'язку в тіні;
- просувайте претендента лише коли він перевершує чемпіона за **незалежними** наслідками (реальні наряди / підтверджені відмови), і зберігайте знімок чемпіона для миттєвого відкату.

Переустановлення не потрібне

Адаптація порогів застосовується на місці через ThresholdUpdateSignal у робочий шлюз. Просування претендента — внутрішня, аудитована заміна стану, а не переустановлення JAR.

10 Розкатка: тінь → рекомендація

- **Тінь (тижні 1–n).** Рекомендації записуються, але не показуються операторам; ви вивчаєте частку хибних спрацювань і даєте базовим рівням усталитися. Зворотний зв'язок усе одно можна збирати з реальних наслідків.
- **Рекомендація (запуск).** Оператори бачать рекомендації й виносять вердикти; цикл зворотного зв'язку починає знижувати зайві тривоги.
- Автономної стадії немає. Модель ніколи не виходить за межі рекомендації.

11 Експлуатація та моніторинг

- Відстежуйте **частку хибних спрацювань за сімейством** і переконайтесь, що вона спадає в міру накопичення зворотного зв'язку.
- Слідкуйте за **зсувом домену / невизначеністю**; зростання означає, що установка відійшла від навчального вікна — заплануйте перепасування (посібник з навчання).

- Робіть знімки збереженого стану нейронів за розкладом, щоб просування можна було відкотити.
- Сигналізуйте про скорочення **запасу часу виявлення** — рекомендації, що спрацювають надто близько до відмови, натякають на потребу уваги до тренду чи порогів.

12 Контрольний список безпеки

- `advisoryOnly` дорівнює `true` й не може бути вимкнено (інваріант `SsMaintConfig`).
- Жоден вихідний шлях не сягає виконавчого механізму; блокування й жорсткий шлюз безпеки поза цією моделлю.
- Зворотний зв'язок автентифіковано й позначено походженням; адаптація замерзає під час зсуву домену.
- Збережений стан має знімки; просування претендентів аудитується й оборотні.

13 Усунення несправностей

Симптом	Ймовірна причина	Виправлення
Швидке падіння на старті	невідповідність <code>neuronnet.classes</code> / збірки	Перегенерувати збірку
Залишки безглузді	неправильне зіставлення <code>тег</code> →сенсор	Узгодити з <code>sensorOrder</code> шапу 1
Пороги не рухаються	зворотний зв'язок не надходить / заморожено	Перевірити підключення й зсув
Навчання втрачено при перезапуску	стан не збережено	Задати записуване <code>storage.json</code>

Розгорніть у тіні, підключіть зворотний зв'язок, дивіться, як частка хибних спрацювань спадає — і ніколи не виводьте це за межі рекомендації. Шар безпеки детермінований і живе поза моделлю, як і має бути.